

Aula 3

Orientação a objetos, SOLID e padrões de projeto

TJSC 2026 - Analista de Sistemas - Banca FGV

Objetivo da aula

Converter sua vivência com Java, Quarkus, Angular, TypeScript, APIs e sistemas corporativos em acertos objetivos no estilo FGV. A aula prioriza conceitos, diferenças finas, pegadinhas e padrões recorrentes em provas recentes de tecnologia da FGV.

Acessibilidade visual aplicada

Este PDF usa texto preto em fundos claros. Tabelas e quadros empregam tons suaves para não prejudicar leitura, impressão ou marcação de estudo.

Sumário

1. Como a FGV vem cobrando este tema
2. Orientação a objetos: fundamentos que viram questão
3. Java e TypeScript: detalhes que a banca pode explorar
4. Coesão, acoplamento, encapsulamento e manutenibilidade
5. SOLID: leitura de prova e aplicação prática
6. GRASP: reforço de alto ROI para questões de responsabilidade
7. Padrões de projeto GoF: categorias e padrões mais prováveis
8. Diferenças que confundem: quadro de pegadinhas
9. Questões autorais no padrão FGV
10. Gabarito resumido
11. Comentários completos
12. Flashcards, mnemônicos e checklist final

1. Como a FGV vem cobrando este tema

A FGV raramente cobra orientação a objetos apenas como vocabulário isolado. Em provas recentes de Analista de Sistemas, a banca mistura definição, exemplo de linguagem, benefício de engenharia, classificação de padrões e julgamento de assertivas. O padrão mais importante para você é: enunciado contextualizado, termos técnicos em inglês, itens I/II/III ou Verdadeiro/Falso e alternativas com conceitos próximos.

Nesta aula, as questões autorais foram desenhadas para simular esse comportamento: enunciados maiores, cenário institucional, termos em inglês expandidos na primeira ocorrência e distratores plausíveis. O objetivo não é adivinhar a prova, mas treinar leitura e discriminação conceitual sob pressão.

Leitura estratégica

Quando a FGV usa um cenário de sistema público, API, microserviço ou módulo Java, ela costuma esconder a pergunta real em uma diferença conceitual: herança versus composição, sobrecarga versus sobrescrita, abstração versus encapsulamento, princípio versus padrão de projeto.

Sinal no enunciado	O que observar	Risco de erro
"classe A estende B"	Herança: A é subclasse/derivada; B é superclasse/base.	Inverter pai e filho.
"subtipo substitui tipo base"	Liskov Substitution Principle - LSP.	Confundir com Open/Closed Principle - OCP.
"depender de abstrações"	Dependency Inversion Principle - DIP.	Confundir com injeção de dependência.
"família de objetos relacionados"	Abstract Factory.	Confundir com Factory Method.
"algoritmo intercambiável em tempo de execução"	Strategy.	Confundir com State.
"notificação automática a interessados"	Observer.	Confundir com publish/subscribe genérico.
"interface incompatível entre classes"	Adapter.	Confundir com Facade.
"interface simplificada para subsistema"	Facade.	Confundir com Adapter.

2. Orientação a objetos: fundamentos que viram questão

Orientação a objetos é um paradigma que organiza software em objetos que possuem identidade, estado e comportamento. Em termos de prova: classe é o molde; objeto é a instância; **atributo representa estado**; **método representa comportamento**; **mensagem é a solicitação de execução de uma operação entre objetos**.

A prova pode tratar Programação Orientada a Objetos - POO, ou Object-Oriented Programming - OOP, como um conjunto de mecanismos para modularidade, reutilização, abstração, encapsulamento, extensão e manutenção. A banca também pode tentar atribuir o benefício correto ao mecanismo errado.

2.1 Classe, objeto, atributo, método, estado e identidade

Termo	Definição objetiva	Exemplo com sistemas corporativos
Classe	Abstração ou tipo que define atributos e métodos comuns.	Classe PagamentoPix em Java.
Objeto	Instância concreta de uma classe em execução.	pagamentoPix123 criado para uma ordem específica.
Atributo	Dado que compõe o estado de um objeto.	valor, chavePix, txid, status.
Método	Operação/comportamento executado por um objeto.	validar(), autorizar(), cancelar().
Estado	Conjunto de valores dos atributos em dado momento.	PENDENTE, AUTORIZADO, REJEITADO.
Identidade	Característica que distingue um objeto de outro.	Dois pagamentos com mesmo valor continuam sendo objetos distintos.

```
public class PagamentoPix {
    private String txid;
    private BigDecimal valor;
    private StatusPagamento status;

    public void autorizar() {
        if (status != StatusPagamento.PENDENTE) {
```

```
        throw new IllegalStateException("Pagamento não está pendente");
    }
    this.status = StatusPagamento.AUTORIZADO;
}
}
```

No código acima, PagamentoPix é uma classe; txid, valor e status são atributos; autorizar() é método; o objeto em execução terá identidade própria e estado variável ao longo do fluxo. Em prova, cuidado para não dizer que atributo é comportamento ou que método é estado.

2.2 Abstração

Abstração é **destacar características essenciais e suprimir detalhes irrelevantes** para um determinado propósito. Em um sistema bancário ou judicial, você não modela todos os detalhes do mundo real; modela os atributos e comportamentos necessários ao caso de uso.

Pegadinha FGV

Abstração não é a mesma coisa que classe abstrata. Classe abstrata é um recurso de linguagem. Abstração é o princípio de modelagem que permite representar o essencial e omitir detalhes.

Exemplo: para julgar se uma guia pode ser paga, talvez baste abstrair valor, vencimento, código de barras, pagador e beneficiário. A cor do papel da guia não entra no modelo porque não afeta o comportamento do sistema.

2.3 Encapsulamento

Encapsulamento consiste em **proteger o estado interno de um objeto** e expor operações controladas para acesso e modificação. Em Java, isso frequentemente aparece com atributos private e métodos públicos que validam invariantes. Em TypeScript, pode aparecer com private, protected, getters, setters e serviços que escondem detalhes de implementação.

```
public class ContaJudicial {
    private BigDecimal saldo;

    public void creditar(BigDecimal valor) {
        if (valor.signum() <= 0) throw new IllegalArgumentException("Valor inválido");
        this.saldo = this.saldo.add(valor);
    }

    public BigDecimal saldoAtual() {
        return saldo;
    }
}
```

A prova costuma associar encapsulamento a ocultação de informação, redução de acoplamento e preservação de invariantes. Porém, encapsulamento não significa esconder toda a classe nem impedir comunicação entre objetos; **significa controlar o acesso ao estado e aos detalhes internos**.

2.4 Herança

Herança cria relação de **especialização/generalização**. Uma subclasse herda membros acessíveis da superclasse e pode especializar comportamento. **Em Java, uma classe possui no máximo uma superclasse direta, mas pode implementar múltiplas interfaces**. Isso é uma pegadinha clássica.

Expressão	Leitura correta
Classe Analista estende Servidor	Analista é subclasse/derivada; Servidor é superclasse/base.
Servidor é pai de Analista	Analista herda características de Servidor.
Interface Autenticavel implementada por UsuarioInterno	UsuarioInterno assume o contrato definido pela interface.
Classe final	Não pode ser estendida.
Método final	Não pode ser sobrescrito.

Herança favorece reutilização e polimorfismo, mas pode aumentar acoplamento se usada indevidamente. Em arquitetura real, especialmente em APIs corporativas e domínios complexos, **composição costuma ser preferível quando o relacionamento não é claramente "é um"**.

2.5 Polimorfismo

Polimorfismo permite que uma mesma operação tenha comportamentos diferentes conforme o tipo concreto do objeto. Em prova, diferencie sobrecarga, sobrescrita e ligação dinâmica.

Conceito	O que significa	Exemplo
Sobrecarga - overloading	Mesmo nome de método, assinaturas diferentes, decisão em tempo de compilação.	calcularTaxa(int dias) e calcularTaxa(LocalDate vencimento).
Sobrescrita - overriding	Subclasse redefine método herdado, respeitando contrato.	PagamentoPix.autorizar() especializa Pagamento.autorizar().
Ligação dinâmica - dynamic binding	Chamada ao método de instância é resolvida conforme o tipo real em tempo de execução.	Servico usa interface Notificador; objeto real é EmailNotificador.
Polimorfismo por subtipo	Objeto de subtipo pode ser tratado como objeto do tipo base.	List<Pagamento> contendo PagamentoPix e PagamentoBoleto.

Regra de prova

Sobrecarga não é herança obrigatoriamente. **Sobrescrita exige relação de herança ou implementação de contrato**. Polimorfismo de subtipo é central para Liskov Substitution Principle - LSP.

2.6 Associação, agregação, composição, dependência e realização

Embora a Aula 2 tenha tratado UML, a FGV pode usar relações de modelagem para testar POO. O ponto de maior retorno é saber diferenciar vínculo fraco, todo-parte e dependência de ciclo de vida.

Relação	Ideia central	Exemplo
Associação	Um objeto conhece ou se relaciona com outro.	Processo possui partes associadas.
Agregação	Todo-parte fraca; a parte pode existir independentemente.	Equipe agrega servidores.
Composição	Todo-parte forte; parte depende do todo para existir no modelo.	Pedido composto por itens do pedido.
Dependência	Uso pontual de outro elemento.	Service depende de DTO em assinatura de método.
Generalização	Herança entre classes.	RecursoJudicial especializa Documento.
Realização	Classe implementa interface.	EmailService realiza Notificador.

3. Java e TypeScript: detalhes que a banca pode explorar

O edital cita Java 17 ou superior em outra aula, mas a FGV pode misturar Java com orientação a objetos. Para esta aula, memorize o núcleo: **Java possui herança simples entre classes; toda classe, exceto Object, tem uma superclasse direta; interfaces permitem múltiplos contratos;** métodos de instância podem ter ligação dinâmica; atributos devem ser encapsulados para preservar invariantes.

Ponto	Java	TypeScript/Angular
Interface	Contrato implementado por classes; pode ter métodos default e static.	Contrato estrutural; muito usado para tipar DTOs, services e responses.
Classe abstrata	Pode conter estado, métodos concretos e abstratos; não é instanciável.	Também existe, mas no Angular é comum usar services e interfaces para contratos.
Herança múltipla de classe	Não permite.	Não permite no modelo clássico de classes.
Composição	Objeto contém/referencia outro para delegar comportamento.	Componente usa services injetados; favorece testabilidade.
Polimorfismo	Interfaces e classes base permitem substituir implementações.	Interfaces, classes e providers permitem substituição em testes.

Conexão com seu trabalho

Quando você injeta um service no Angular ou um bean em Quarkus, você está frequentemente praticando uma variação de composição e inversão de dependência: o consumidor não precisa construir diretamente a implementação concreta.

4. Coesão, acoplamento, encapsulamento e manutenibilidade

Coesão mede o quanto as responsabilidades internas de um módulo pertencem a uma mesma finalidade. Acoplamento **mede o grau de dependência entre módulos.** Em engenharia de software, geralmente se busca alta coesão e baixo acoplamento.

Conceito	Desejável	Indício ruim	Como a FGV pode cobrar
Coesão	Alta	Classe faz validação, persistência, envio de e-mail e regra fiscal ao mesmo tempo.	Associar Single Responsibility Principle - SRP a alta coesão.
Acoplamento	Baixo	Service cria diretamente new RepositorioOracle() em vários pontos.	Associar Dependency Inversion Principle - DIP a dependência de abstrações.
Encapsulamento	Forte	Atributos públicos alterados sem validação.	Ocultação de informação e controle de acesso.
Manutenibilidade	Alta	Mudanças locais e previsíveis.	Princípios e padrões como meios, não fins.

5. SOLID: leitura de prova e aplicação prática

SOLID é um acrônimo para cinco princípios de design orientado a objetos. Em prova, a banca costuma pedir a letra associada à definição ou apresentar um cenário e perguntar qual princípio foi violado. Em projetos reais, SOLID **melhora manutenibilidade e testabilidade** quando aplicado com bom senso; usado de forma dogmática, pode gerar complexidade desnecessária.

Letra	Nome expandido	Ideia de prova	Sinal típico no enunciado
S	Single Responsibility Principle - Princípio da Responsabilidade Única	Uma classe deve ter uma razão principal para mudar.	Classe concentrando responsabilidades de regra, persistência, log, notificação.

O	Open/Closed Principle - Princípio Aberto/Fechado	Aberto para extensão, fechado para modificação.	Novos tipos exigem alterar switch/if central repetidamente.
L	Liskov Substitution Principle - Princípio da Substituição de Liskov	Subtipos devem poder substituir o tipo base sem quebrar propriedades esperadas.	Subclasse viola contrato, lança exceções inesperadas ou enfraquece pós-condições.
I	Interface Segregation Principle - Princípio da Segregação de Interfaces	Clientes não devem depender de métodos que não usam.	Interface gigante obriga classes a implementar métodos vazios.
D	Dependency Inversion Principle - Princípio da Inversão de Dependência	Módulos de alto nível dependem de abstrações, não de detalhes concretos.	Service depende diretamente de classe concreta de infraestrutura.

5.1 S - Single Responsibility Principle

O SRP não significa que uma classe deve ter apenas um método. Significa que deve possuir uma responsabilidade coesa e uma razão principal de mudança. Em um sistema corporativo, uma classe `ProcessadorPagamento` que valida regra de negócio, monta SQL, envia e-mail, chama API externa e gera PDF provavelmente viola SRP.

```
// Ruim: muitas razões para mudar
class PagamentoService {
    void validar() {}
    void salvarNoOracle() {}
    void enviarEmail() {}
    void gerarPdf() {}
}

// Melhor: responsabilidades separadas
class PagamentoService { void processar() {} }
class PagamentoRepository { void salvar() {} }
class NotificadorPagamento { void enviar() {} }
```

5.2 O - Open/Closed Principle

O OCP exige que o comportamento seja ampliado com novas implementações, sem alterar continuamente código já testado. Na prática, `Strategy` e `Factory Method` costumam apoiar esse princípio.

```
interface CalculadoraTarifa {
    BigDecimal calcular(Pagamento pagamento);
}

class TarifaPix implements CalculadoraTarifa { ... }
class TarifaBoleto implements CalculadoraTarifa { ... }
// Novo tipo: cria nova classe, sem alterar um switch central enorme.
```

Pegadinha

OCP não proíbe qualquer modificação no sistema. Ele reduz a necessidade de modificar módulos estáveis quando novos comportamentos são adicionados.

5.3 L - Liskov Substitution Principle

O LSP trata substitutibilidade. Se uma rotina aceita um tipo base, ela deve funcionar corretamente com seus subtipos. A FGV já cobrou definição formal do tipo: se uma propriedade vale para objetos do tipo T, também deve valer para objetos do subtipo S.

Exemplo clássico: se Retângulo permite setLargura e setAltura independentes, Quadrado como subtipo pode quebrar expectativas ao forçar largura e altura iguais. O erro não é usar herança em si; é modelar subtipo que não preserva contrato comportamental.

5.4 I - Interface Segregation Principle

O ISP combate interfaces gordas. Uma classe não deve ser forçada a depender de **métodos que não usa**. Em uma API corporativa, em vez de uma interface DocumentoService com assinar(), arquivar(), digitalizar(), protocolar(), expurgar() e restaurar(), talvez faça sentido separar contratos específicos.

```
interface AssinadorDocumento { void assinar(Documento doc); }
interface ArquivadorDocumento { void arquivar(Documento doc); }
interface RestauradorDocumento { void restaurar(Documento doc); }
```

5.5 D - Dependency Inversion Principle

O DIP diz que módulos de alto nível não devem depender de módulos de baixo nível; ambos devem depender de abstrações. Abstrações não devem depender de detalhes; detalhes devem depender de abstrações. Em Quarkus, CDI - Contexts and Dependency Injection - ajuda a implementar essa ideia, mas DIP não é sinônimo de framework de injeção.

```
interface RepositorioProcesso {
    Processo buscarPorNumero(String numero);
}

class ProcessoService {
    private final RepositorioProcesso repositorio;
    ProcessoService(RepositorioProcesso repositorio) {
        this.repositorio = repositorio;
    }
}
```

DIP versus DI

Dependency Inversion Principle - DIP é princípio de design. Dependency Injection - DI é técnica para fornecer dependências. Um framework pode facilitar DI, mas não garante bom design por si só.

6. GRASP: reforço de alto ROI para questões de responsabilidade

GRASP - General Responsibility Assignment Software Principles - não está nominalmente na trilha atual, mas aparece em provas FGV junto com SOLID e padrões de projeto. Vale dominar o núcleo porque a cobrança é curta e conceitual: quem deve receber determinada responsabilidade?

Princípio GRASP	Ideia central	Pegadinha
Information Expert	Atribuir responsabilidade à classe que possui as informações necessárias.	Não é necessariamente a classe controladora nem a interface gráfica.
Creator	Uma classe deve criar objetos que	Não é para criar tudo em um único

	contém, agrega ou usa intensamente.	Factory global.
Controller	Objeto de controle recebe eventos de sistema e coordena casos de uso.	Controller não deve conter toda a regra de negócio.
Low Coupling	Reduzir dependências entre classes.	Não significa ausência total de dependência.
High Cohesion	Manter responsabilidades relacionadas no mesmo módulo.	Não significa classes minúsculas sem propósito.
Polymorphism	Usar polimorfismo para variar comportamento.	Evita condicionais por tipo espalhadas.
Protected Variations	Proteger pontos de variação por interfaces ou abstrações.	Conecta-se ao OCP e DIP.

7. Padrões de projeto GoF: categorias e padrões mais prováveis

Padrões de projeto são **soluções recorrentes para problemas recorrentes de design**. O grupo conhecido como GoF - Gang of Four - organizou padrões em três categorias: criacionais, estruturais e comportamentais. A FGV gosta de cobrar classificação, intenção do padrão e confusão entre padrões parecidos.

Categoria	Pergunta que responde	Padrões GoF
Criacionais	Como criar objetos sem acoplar demais o cliente à classe concreta?	Abstract Factory, Builder, Factory Method, Prototype, Singleton.
Estruturais	Como compor classes e objetos para formar estruturas flexíveis?	Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy.
Comportamentais	Como distribuir responsabilidades e comunicação entre objetos?	Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor.

7.1 Padrões criacionais

Padrão	Intenção	Exemplo prático
Singleton	Garantir uma única instância e ponto global de acesso controlado.	Configuração compartilhada, mas cuidado com testes e concorrência.
Factory Method	Delegar a subclasses ou métodos especializados a criação do objeto.	Criar validador conforme tipo de documento.
Abstract Factory	Criar famílias de objetos relacionados sem especificar classes concretas.	Componentes UI para tema desktop/mobile ou clientes de serviços compatíveis.
Builder	Construir objeto complexo passo a passo.	Montagem de requisição complexa com campos opcionais.
Prototype	Criar objetos copiando protótipos existentes.	Clonar configuração base de workflow.

Factory Method x Abstract Factory

Factory Method foca um método de criação e costuma lidar com um produto. Abstract Factory cria famílias de produtos relacionados. A FGV adora aproximar esses dois.

7.2 Padrões estruturais

Padrão	Intenção	Exemplo prático
Adapter	Converter interface de uma classe em outra esperada pelo cliente.	Adaptar cliente SOAP legado para interface interna REST-like.
Facade	Fornecer interface simplificada para subsistema complexo.	ServicoPagamentoFacade orquestra validação, antifraude e autorização.
Decorator	Adicionar responsabilidades dinamicamente sem alterar a classe original.	Adicionar log, auditoria ou compressão em cadeia.
Proxy	Controlar acesso a outro objeto.	Proxy remoto, proxy de segurança ou

		lazy loading.
Composite	Tratar objetos individuais e composições de forma uniforme.	Árvore de unidades, menus ou documentos compostos.
Bridge	Separar abstração de implementação para variar independentemente.	Canal de notificação separado do conteúdo.
Flyweight	Compartilhar objetos pequenos e repetidos para economizar memória.	Ícones, estilos, objetos imutáveis repetidos.

Adapter x Facade

Adapter altera compatibilidade de interface entre cliente e classe existente. Facade simplifica acesso a um subsistema. Adapter resolve incompatibilidade; Facade reduz complexidade de uso.

7.3 Padrões comportamentais

Padrão	Intenção	Exemplo prático
Strategy	Encapsular algoritmos intercambiáveis.	Cálculo de tarifa Pix, boleto ou convênio.
State	Alterar comportamento conforme estado interno.	Processo muda operações permitidas em rascunho, protocolado, arquivado.
Observer	Notificar interessados quando ocorre mudança de estado.	Publicar evento de pagamento autorizado para notificadores.
Template Method	Definir esqueleto de algoritmo e deixar passos para subclasses.	Fluxo padrão de validação com etapas especializadas.
Command	Encapsular solicitação como objeto.	Comando de estornar, cancelar, reprocessar.
Chain of Responsibility	Passar requisição por cadeia de tratadores.	Validações sucessivas em ordem.
Iterator	Percorrer coleção sem expor representação interna.	Iterar documentos de um processo.
Mediator	Centralizar comunicação entre objetos.	Mediador entre componentes de uma tela complexa.
Memento	Capturar e restaurar estado sem violar encapsulamento.	Desfazer edição de documento.
Visitor	Adicionar operação a estrutura de objetos sem alterar suas classes.	Exportadores/relatórios sobre árvore de elementos.

Strategy x State

Strategy seleciona algoritmo intercambiável, geralmente por política externa. State muda comportamento conforme estado interno do objeto, parecendo trocar sua classe. Em prova, observe quem dirige a variação: escolha de algoritmo ou ciclo de estados.

8. Diferenças que confundem: quadro de pegadinhas

Confusão comum	Como resolver na prova
Abstração x encapsulamento	Abstração escolhe o essencial; encapsulamento protege detalhes internos e estado.
Herança x composição	Herança é relação "é um"; composição é relação "tem um/usa um".
Sobrecarga x sobrescrita	Sobrecarga muda assinatura; sobrescrita redefine comportamento herdado.
LSP x polimorfismo	Polimorfismo permite substituição; LSP exige que a substituição preserve contrato.
DIP x DI	DIP é princípio; DI é técnica.
OCP x Strategy	OCP é princípio; Strategy é padrão que pode ajudar a cumpri-lo.
ISP x SRP	ISP trata interfaces específicas para clientes; SRP trata razão de mudança de classe/módulo.
Adapter x Facade	Adapter compatibiliza interfaces; Facade simplifica

	subsistema.
Decorator x Proxy	Decorator adiciona comportamento; Proxy controla acesso.
Factory Method x Abstract Factory	Factory Method cria produto; Abstract Factory cria família de produtos relacionados.
Singleton x objeto estático global	Singleton controla instância; uso excessivo pode prejudicar testes e acoplamento.

8.1 Herança ou composição: decisão de prova e de projeto

A FGV pode apresentar um cenário em que a resposta tecnicamente correta não é simplesmente "use herança". Em sistemas reais, **herança deve representar uma relação estável de especialização**: um Analista é um Servidor; um PagamentoPix é um Pagamento, se todos os contratos de Pagamento forem preservados. Composição deve ser preferida quando um objeto usa outro para cumprir uma tarefa: ProcessoService tem um ProcessoRepository; Pagamento tem uma PoliticaTarifa; Documento usa um AssinadorDigital.

Pergunta mental	Tendência de resposta	Exemplo
O objeto especializado realmente é um tipo do objeto base?	Herança pode fazer sentido.	RecursoEspecial é um tipo de Recurso.
Quero trocar comportamento em tempo de execução?	Composição/Strategy tende a ser melhor.	Trocar política de tarifa sem alterar Pagamento.
A subclasse quebra promessa da superclasse?	Evite herança; risco de violar LSP.	DocumentoTemporario que não pode arquivar se Documento promete arquivar.
O relacionamento é todo-parte forte?	Composição no sentido de modelagem.	Pedido e ItemPedido no mesmo ciclo de vida.
A classe só precisa usar um serviço externo?	Dependência/composição, não herança.	ProcessoService usa Notificador.

Atalho para prova

Quando a alternativa diz que herança **sempre aumenta segurança, sempre reduz acoplamento ou sempre melhora manutenção, desconfie**. Herança pode aumentar reutilização, mas também pode acoplar fortemente a subclasse à superclasse.

8.2 Classes abstratas, interfaces, records, final e sealed em Java

Embora a aula de linguagens venha depois, alguns detalhes de Java 17+ são naturalmente cobrados junto de orientação a objetos. A FGV já vem usando enunciados com código e termos como sealed, final e interface. O foco aqui é conceitual, não decoreba de sintaxe avançada.

Recurso Java	Ideia central	Pegadinha provável
interface	Define contrato; pode ser implementada por várias classes.	Não é classe concreta; uma classe pode implementar múltiplas interfaces.
abstract class	Classe não instanciável, pode conter estado e métodos concretos/abstratos.	Não confundir abstração como princípio com classe abstrata como recurso.
final class	Classe que não pode ser estendida.	final em classe não é o mesmo que final em variável.
final method	Método que não pode ser sobrescrito.	Pode existir em classe não final.
record	Classe especial para dados imutáveis/transparentes em Java moderno.	Não é substituto universal de entidade rica de domínio.
sealed class	Restringe quais classes podem estender/implementar.	As subclasses permitidas devem obedecer regras como final, sealed ou non-sealed.
static method	Método associado à classe, não à instância.	Não participa de sobrescrita polimórfica como método de instância.

```
public sealed abstract class Documento permits Peticao, Sentenca {}
```

```

public final class Peticao extends Documento {}
public non-sealed class Sentenca extends Documento {}

public interface Assinavel {
    void assinar();
}

```

No exemplo, Documento restringe explicitamente a hierarquia. Peticao não pode ser estendida por ser final. Sentenca libera extensão por ser non-sealed. Esse tipo de detalhe pode aparecer como questão de código, mas a ideia central continua sendo controle de herança e contrato de tipos.

8.3 Cheiros de enunciado FGV para padrões GoF

Em vez de decorar apenas nomes, treine a identificar a "necessidade" descrita. A FGV costuma redigir o problema e espera que o candidato reconheça o padrão pela intenção.

Expressão ou necessidade no enunciado	Padrão provável	Categoria
Garantir uma única instância controlada	Singleton	Criacional
Criar objeto sem acoplar cliente à classe concreta	Factory Method	Criacional
Criar famílias de objetos relacionados	Abstract Factory	Criacional
Construir objeto complexo passo a passo	Builder	Criacional
Copiar objeto protótipo existente	Prototype	Criacional
Compatibilizar interface incompatível	Adapter	Estrutural
Simplificar acesso a subsistema complexo	Facade	Estrutural
Adicionar comportamento sem alterar classe original	Decorator	Estrutural
Controlar acesso, lazy loading, remoto ou proteção	Proxy	Estrutural
Tratar parte e todo de forma uniforme	Composite	Estrutural

Expressão ou necessidade no enunciado	Padrão provável	Categoria
Algoritmos intercambiáveis	Strategy	Comportamental
Comportamento varia conforme estado interno	State	Comportamental
Interessados recebem notificação de mudança	Observer	Comportamental
Esqueleto de algoritmo fixo com etapas variáveis	Template Method	Comportamental
Solicitação encapsulada como objeto	Command	Comportamental
Requisição percorre cadeia de tratadores	Chain of Responsibility	Comportamental
Percorrer coleção sem expor estrutura	Iterator	Comportamental
Centralizar comunicação entre objetos	Mediator	Comportamental
Salvar/restaurar estado sem violar encapsulamento	Memento	Comportamental
Adicionar operação a estrutura de objetos	Visitor	Comportamental

8.4 Padrões versus arquitetura, framework e recurso de linguagem

Outra pegadinha é misturar níveis de abstração. MVC - Model-View-Controller - é padrão arquitetural ou padrão de apresentação, não um padrão GoF clássico. Spring, Quarkus e Angular são

frameworks/plataformas, não padrões de projeto. interface, abstract, final e sealed são recursos de linguagem, não padrões GoF. SOLID é conjunto de princípios, não catálogo de padrões.

Item	Natureza	Como não cair
SOLID	Princípios de design	Não é padrão GoF. Pode ser apoiado por padrões.
GRASP	Princípios de atribuição de responsabilidade	Não é categoria GoF.
Factory Method	Padrão GoF criacional	Não é apenas qualquer método que cria objeto.
MVC	Padrão arquitetural/apresentação	Não é um dos 23 GoF clássicos.
Quarkus CDI	Framework/mecanismo de injeção	Pode implementar DI e apoiar DIP, mas não é DIP em si.
Java interface	Recurso de linguagem/contrato	Pode viabilizar DIP, ISP e polimorfismo.
REST	Estilo arquitetural	Não é padrão GoF.

8.5 Quadro de "certo, errado e cuidado"

Afirmção de prova	Julgamento	Comentário
Encapsulamento impede totalmente que objetos se comuniquem.	Errado	Ele controla acesso; objetos continuam colaborando por interfaces públicas.
Abstração pode ser aplicada sem classe abstrata.	Certo	É princípio de modelagem; classe abstrata é apenas um recurso possível.
Toda herança melhora manutenibilidade.	Errado	Herança inadequada pode aumentar acoplamento e violar LSP.
Java não permite herança múltipla de classes.	Certo	Permite implementar múltiplas interfaces.
Sobrescrita depende de relação de herança/contrato.	Certo	Subclasse redefine comportamento de método herdado ou implementado.
Strategy e State são idênticos.	Errado	Strategy varia algoritmo; State varia comportamento conforme estado interno.
Adapter e Facade são estruturais.	Certo	Mas resolvem problemas diferentes.
SOLID e GoF são mutuamente exclusivos.	Errado	Podem ser usados em conjunto.
DIP é igual a injeção de dependência.	Cuidado	DI é técnica; DIP é princípio.

8.6 Aplicação no seu contexto Java/Quarkus/Angular

Para converter sua experiência prática em acerto de prova, pense sempre em duas camadas: a camada da prática e a camada da definição. No trabalho, você vê CDI, services, repositories, DTOs, controllers, interceptors e integração com APIs. Na prova, a FGV quer saber qual princípio ou padrão está por trás da decisão.

Situação prática	Tradução para prova
Service recebe Repository por construtor/injeção	Composição, dependência de abstração, possível DIP.
Controller apenas coordena request e chama service	SRP e separação de responsabilidades.
Interface Notificador com EmailNotificador e TeamsNotificador	Polimorfismo, OCP e Strategy em alguns contextos.
Interceptor adiciona log/auditoria a chamadas	Pode lembrar Decorator/Proxy dependendo da intenção.
Adapter para cliente externo SOAP	Adapter se o foco for compatibilizar interface.
Facade para orquestrar subsistema de pagamentos	Facade se o foco for simplificar acesso a vários componentes.
State machine de pagamento	State se o comportamento depende do estado interno.
Eventos publicados após alteração de status	Observer/event-driven; cuidado para não chamar tudo de Observer sem relação sujeito-observador.

Como responder FGV

Não responda pelo nome da tecnologia que você usa no trabalho. Responda pela intenção conceitual:

criar, adaptar, simplificar, notificar, trocar algoritmo, variar por estado, proteger estado interno, reduzir acoplamento ou atribuir responsabilidade.

9. Hacks de prova e inferência por inglês técnico

- Single Responsibility: single = uma responsabilidade central / uma razão principal para mudar.
- Open/Closed: aberto para extensão, fechado para modificação de código estável.
- Liskov Substitution: substitution = substituição segura de subtipo por tipo base.
- Interface Segregation: segregation = separar interfaces grandes em contratos menores.
- Dependency Inversion: inversion = inverter dependência de concreto para abstração.
- Factory = fábrica/criação; Strategy = estratégia/algoritmo; Observer = observador/notificação.
- Adapter = adaptador para compatibilidade; Facade = fachada simplificadora; Decorator = decoração/adiciona comportamento.
- Composite = composição em árvore; State = estado interno; Command = comando como objeto.

10. Mapa mental textual

OO -> classe/objeto -> estado/comportamento/identidade ->
abstração/encapsulamento/herança/polimorfismo.

Design bom -> alta coesão + baixo acoplamento + encapsulamento + testes fáceis + mudanças localizadas.

SOLID -> SRP (responsabilidade) -> OCP (extensão) -> LSP (substituição) -> ISP (interfaces pequenas) -> DIP (abstrações).

GoF -> Criacionais (criação) -> Estruturais (composição/estrutura) -> Comportamentais
(colaboração/algoritmo/comunicação).

FGV -> cenário + assertivas + conceitos próximos + alternativa correta por discriminação fina.

11. Questões autorais - estilo FGV

Instrução

Resolva as 20 questões antes de consultar o gabarito. Nesta seção não há marcação de resposta, comentário ou pista deliberada.

Questão 1

No desenvolvimento de um sistema corporativo para **controle de ordens de pagamento**, uma equipe modelou a classe `OrdemPagamento` com atributos como valor, favorecido, data de vencimento e situação atual. A classe também possui operações como `validar()`, `autorizar()` e `cancelar()`. Durante uma revisão técnica, um analista afirmou que o objetivo dessa modelagem foi representar apenas os elementos relevantes para o domínio de pagamentos, omitindo detalhes do mundo real que não interferem no comportamento do sistema, como a forma visual do documento físico que originou a ordem.

No contexto de Programação Orientada a Objetos - POO (Object-Oriented Programming - OOP), a afirmação do analista descreve principalmente o conceito de

- (A) encapsulamento, pois os atributos são necessariamente privados e acessíveis apenas por métodos públicos.
- (B) herança, pois a classe OrdemPagamento deve ser derivada de uma classe genérica DocumentoFisico.
- (C) abstração, pois foram selecionados os aspectos essenciais do domínio e omitidos detalhes irrelevantes para o propósito do sistema.
- (D) polimorfismo, pois os métodos validar(), autorizar() e cancelar() possuem obrigatoriamente múltiplas implementações.
- (E) composição, pois todo objeto de domínio deve ser formado por outros objetos com ciclo de vida dependente.

Questão 2

Em uma aplicação Java 17 de um órgão público, a classe ServidorPublico possui atributos protegidos e métodos de negócio comuns. A classe AnalistaSistemas foi declarada como extensão de ServidorPublico, sem uso de interfaces no trecho analisado. Um estagiário escreveu em um relatório que "ServidorPublico deriva de AnalistaSistemas, pois a classe mais geral sempre é criada a partir da mais específica".

Considerando a terminologia usual de orientação a objetos e a linguagem Java, assinale a afirmativa correta.

- (A) AnalistaSistemas é subclasse ou classe derivada de ServidorPublico, enquanto ServidorPublico é superclasse ou classe base.
- (B) ServidorPublico é subclasse de AnalistaSistemas, pois concentra os atributos herdados pela classe especializada.
- (C) AnalistaSistemas e ServidorPublico são classes irmãs, pois Java não permite relação direta de herança entre classes concretas.
- (D) AnalistaSistemas é interface de ServidorPublico, pois toda extensão em Java representa implementação de contrato.
- (E) ServidorPublico é composição de AnalistaSistemas, pois a classe geral contém obrigatoriamente a classe específica.

Questão 3

Uma equipe que mantém uma API interna observou que várias classes acessavam diretamente atributos públicos de uma entidade de domínio, alterando seu status sem validação. Em alguns fluxos, uma ordem já liquidada voltava indevidamente para pendente porque outro módulo atribuiu diretamente o valor do campo situacao. A proposta de correção foi tornar o estado interno inacessível diretamente e expor operações como marcarComoLiquidada() e reabrirComJustificativa(), com validações internas.

A proposta está associada principalmente ao princípio de

- (A) herança múltipla, pois a entidade passará a herdar validações de várias classes simultaneamente.
- (B) sobrecarga de métodos, pois todo atributo privado deve possuir vários métodos com o mesmo nome.
- (C) abstração funcional, pois todo detalhe de estado deve ser eliminado do objeto.
- (D) polimorfismo paramétrico, pois a situação deve ser representada por tipos genéricos.

(E) encapsulamento, pois o acesso ao estado interno passa a ser controlado por operações que preservam invariantes.

Questão 4

No módulo de cálculo de custas de um sistema judicial, há uma interface `CalculadoraCusta` com o método `calcular(Processo p)`. As classes `CalculadoraCustaCivel`, `CalculadoraCustaCriminal` e `CalculadoraCustaExecucaoFiscal` implementam essa interface com regras distintas. O serviço de orquestração recebe uma `CalculadoraCusta` e invoca `calcular(p)`, sem conhecer a classe concreta utilizada em tempo de execução.

A situação descrita exemplifica principalmente

- (A) agregação, pois todas as calculadoras possuem ciclo de vida dependente do processo.
- (B) polimorfismo, pois uma mesma operação definida por contrato pode apresentar comportamento diferente conforme a implementação concreta.
- (C) sobrecarga, pois o método `calcular` possui obrigatoriamente várias assinaturas diferentes.
- (D) composição, pois a interface `CalculadoraCusta` é parte indivisível de `Processo`.
- (E) serialização, pois as regras de cálculo são transformadas em bytes para transmissão.

Questão 5

Em Java, uma classe `RelatorioProcessual` possui dois métodos denominados `gerar`: `gerar(Processo p)` e `gerar(Processo p, FormatoRelatorio formato)`. Em outra parte do sistema, a classe `RelatorioProcessualPDF` estende `RelatorioProcessual` e redefine o método `gerar(Processo p)` para produzir um relatório com formatação específica.

Assinale a opção que classifica corretamente os dois fenômenos descritos.

- (A) Ambos são sobrescrita, pois o nome do método é o mesmo nos dois casos.
- (B) Ambos são sobrecarga, pois todo método com mesmo nome em Java é escolhido apenas em tempo de compilação.
- (C) O primeiro é sobrescrita e o segundo é composição.
- (D) O primeiro é sobrecarga e o segundo é sobrescrita.
- (E) O primeiro é polimorfismo paramétrico e o segundo é agregação.

Questão 6

Em uma revisão de arquitetura, foi observado que a classe `GerenciadorProcesso` realizava validação de regras processuais, persistência em banco, montagem de mensagens para filas, envio de e-mails e geração de documentos PDF. Cada alteração em qualquer uma dessas políticas exigia recompilar e testar a mesma classe. O arquiteto propôs dividir essas responsabilidades em componentes mais coesos.

A recomendação do arquiteto está mais diretamente relacionada ao princípio

- (A) Single Responsibility Principle - SRP, pois a classe possuía múltiplas razões para mudar.
- (B) Open/Closed Principle - OCP, pois a classe não utilizava obrigatoriamente herança múltipla.

(C) Liskov Substitution Principle - LSP, pois objetos de subclasse não podiam substituir objetos de superclasse.

(D) Interface Segregation Principle - ISP, pois o problema central era uma interface com métodos não utilizados.

(E) Dependency Inversion Principle - DIP, pois toda classe concreta deve ser proibida em sistemas orientados a objetos.

Questão 7

Uma equipe implementou uma rotina de cálculo de prioridade para filas de atendimento. A cada novo tipo de prioridade, os desenvolvedores precisavam alterar um grande bloco switch em uma classe central já estabilizada. Após incidentes em produção, a equipe substituiu a estrutura por uma interface PolíticaPrioridade e por implementações específicas, permitindo adicionar novas políticas com menor alteração no código central.

Essa refatoração aplica principalmente o princípio

(A) Single Responsibility Principle - SRP, porque toda classe passa a ter exatamente um método público.

(B) Liskov Substitution Principle - LSP, porque toda política concreta passa a herdar obrigatoriamente estado da classe central.

(C) Interface Segregation Principle - ISP, porque uma interface deve sempre possuir apenas um método.

(D) Dependency Inversion Principle - DIP, porque a aplicação deixa de possuir módulos de alto nível.

(E) Open/Closed Principle - OCP, porque o comportamento passa a ser ampliado por extensão, reduzindo alterações em código estável.

Questão 8

Em um sistema de gestão documental, a classe Documento possui o método arquivar(), cujo contrato estabelece que documentos válidos podem ser arquivados sem lançar exceção de operação não suportada. A subclasse DocumentoTemporario foi criada como especialização de Documento, mas sua implementação de arquivar() sempre lança UnsupportedOperationException, ainda que o documento esteja válido, porque documentos temporários não deveriam ser arquiváveis.

Considerando os princípios SOLID, há violação mais direta do

(A) Single Responsibility Principle - SRP, porque Documento possui apenas um método de negócio.

(B) Open/Closed Principle - OCP, porque DocumentoTemporario foi criado por extensão.

(C) Liskov Substitution Principle - LSP, porque o subtipo não preserva o comportamento esperado para o tipo base.

(D) Interface Segregation Principle - ISP, porque toda classe concreta implementa métodos públicos.

(E) Dependency Inversion Principle - DIP, porque a exceção lançada é uma classe concreta da plataforma Java.

Questão 9

Uma interface `ServicoDocumento` foi criada com os métodos `assinar()`, `digitalizar()`, `arquivar()`, `restaurar()`, `expurgar()`, `enviarParaFila()` e `gerarEtiqueta()`. Classes que apenas assinavam documentos foram obrigadas a implementar métodos de digitalização e expurgo com corpo vazio ou exceções artificiais. A equipe decidiu decompor essa interface em contratos menores, como `AssinadorDocumento`, `ArquivadorDocumento` e `DigitalizadorDocumento`.

A decisão está mais relacionada ao

- (A) Open/Closed Principle - OCP, pois novas classes passam a ser proibidas.
- (B) Interface Segregation Principle - ISP, pois clientes não devem depender de métodos que não utilizam.
- (C) Liskov Substitution Principle - LSP, pois toda interface deve possuir uma hierarquia de classes abstratas.
- (D) Dependency Inversion Principle - DIP, pois interfaces nunca podem declarar mais de um método.
- (E) Single Responsibility Principle - SRP, pois interfaces não podem representar capacidades do domínio.

Questão 10

Em uma aplicação Quarkus, a classe `ProcessoService` instanciava diretamente `OracleProcessoRepository` com o operador `new`. Isso dificultava testes unitários e acoplava a regra de negócio a uma tecnologia de persistência. A equipe criou a interface `ProcessoRepository` e passou a receber uma implementação por injeção de dependência, permitindo substituir a infraestrutura por repositório em memória durante testes.

Do ponto de vista de SOLID, a mudança se alinha principalmente ao

- (A) Single Responsibility Principle - SRP, porque todo service deve possuir apenas um construtor.
- (B) Open/Closed Principle - OCP, porque o banco Oracle passa a ser a única extensão permitida.
- (C) Liskov Substitution Principle - LSP, porque repositórios não podem possuir métodos públicos.
- (D) Dependency Inversion Principle - DIP, porque o módulo de alto nível passa a depender de abstração, e não diretamente de detalhe concreto.
- (E) Interface Segregation Principle - ISP, porque uma interface de repositório deve ser vazia para não criar acoplamento.

Questão 11

Em uma prova de conceito, uma equipe decidiu utilizar padrões GoF (Gang of Four) para melhorar o desenho de um módulo de integração. Um analista afirmou que Singleton, Factory Method, Builder, Adapter, Facade, Strategy e Observer pertencem todos à mesma categoria, pois todos são padrões de projeto orientados a objetos.

Considerando a classificação tradicional dos padrões GoF, a afirmação do analista está

- (A) correta, pois todos os padrões GoF são criacionais.
- (B) correta, pois todos os padrões GoF são estruturais.
- (C) correta, pois todos os padrões GoF são comportamentais.

(D) incorreta apenas quanto a Observer, que não é padrão GoF.

(E) incorreta, pois há padrões criacionais, estruturais e comportamentais na lista.

Questão 12

Um módulo precisa criar diferentes validadores de documento conforme o tipo de processo, mas o código cliente não deve conhecer diretamente as classes concretas ValidadorCivel, ValidadorCriminal e ValidadorExecucaoFiscal. A solução adotada define um método de criação que retorna o validador adequado, encapsulando a decisão de instanciação e reduzindo o acoplamento do cliente às classes concretas.

O padrão de projeto mais diretamente descrito é

(A) Factory Method, pois a criação do objeto é delegada/encapsulada em um método de fábrica.

(B) Observer, pois os validadores devem notificar automaticamente todos os processos cadastrados.

(C) Decorator, pois o objetivo central é adicionar comportamento dinamicamente a um objeto já existente.

(D) Composite, pois o validador individual e o conjunto de validadores devem ser tratados uniformemente em árvore.

(E) Memento, pois o objetivo é capturar e restaurar o estado interno do validador.

Questão 13

Um sistema possui clientes externos SOAP (Simple Object Access Protocol) legados, mas internamente a equipe deseja expor uma interface mais simples para as regras de negócio, com métodos como consultarSituacao() e registrarEvento(). Foi criada uma classe que recebe chamadas no formato esperado pelo novo módulo e as traduz para chamadas compatíveis com a biblioteca SOAP legada, cuja interface não pode ser alterada.

O padrão de projeto mais adequado para descrever essa solução é

(A) Facade, pois toda simplificação de acesso a sistema legado é, por definição, uma fachada.

(B) Proxy, pois qualquer chamada a serviço externo deve controlar acesso remoto.

(C) Adapter, pois há conversão de uma interface existente para outra esperada pelo cliente.

(D) Builder, pois a solução constrói objetos complexos passo a passo.

(E) Strategy, pois o objetivo é selecionar algoritmos intercambiáveis de cálculo.

Questão 14

Em uma arquitetura corporativa, o subsistema de pagamento envolve validação cadastral, consulta antifraude, reserva de saldo, autorização, emissão de comprovante e envio de eventos. Para evitar que cada caso de uso conheça todos esses componentes e sua ordem de chamada, foi criada uma classe PagamentoFacade com operações de alto nível como autorizarPagamento() e cancelarPagamento().

O padrão de projeto descrito é

(A) Adapter, pois converte obrigatoriamente a interface de uma classe incompatível em outra.

(B) Facade, pois fornece uma interface simplificada para um subsistema complexo.

- (C) Observer, pois todos os componentes internos são notificados por evento.
- (D) Prototype, pois cria novos pagamentos por clonagem de instâncias existentes.
- (E) Flyweight, pois compartilha objetos pequenos para reduzir consumo de memória.

Questão 15

Um serviço calcula descontos em custas processuais. Existem políticas diferentes: desconto por convênio, desconto por faixa de renda, desconto por mutirão e ausência de desconto. A equipe deseja trocar a política em tempo de execução, sem espalhar condicionais por tipo de desconto em vários pontos do sistema.

O padrão de projeto que melhor atende a esse objetivo é

- (A) Singleton, por garantir uma única instância global de cada desconto.
- (B) Memento, por permitir desfazer o desconto aplicado.
- (C) Composite, por obrigar descontos a serem organizados em árvore.
- (D) Strategy, por encapsular algoritmos intercambiáveis sob uma interface comum.
- (E) Visitor, por impedir alterações nas classes visitadas durante o cálculo.

Questão 16

Um objeto `ProcessoAdministrativo` possui estados internos como `RASCUNHO`, `PROTOCOLADO`, `EM_ANALISE`, `ARQUIVADO` e `CANCELADO`. As operações permitidas variam de acordo com o estado; por exemplo, `cancelar()` é permitido em `RASCUNHO` e `PROTOCOLADO`, mas não em `ARQUIVADO`. A equipe decidiu representar cada estado por uma classe com comportamento próprio, evitando uma sequência extensa de condicionais no objeto principal.

O padrão de projeto mais adequado é

- (A) Strategy, pois qualquer variação de comportamento por classe é sempre uma estratégia.
- (B) Template Method, pois os estados são passos fixos de um algoritmo definido na superclasse.
- (C) State, pois o comportamento do objeto varia conforme seu estado interno.
- (D) Abstract Factory, pois cada estado cria uma família de objetos relacionados.
- (E) Bridge, pois estados e implementações devem variar de forma independente em duas hierarquias.

Questão 17

Após a autorização de uma ordem de pagamento, diferentes componentes precisam reagir: um envia notificação ao usuário, outro registra auditoria, outro publica evento para integração e outro atualiza painel operacional. A equipe deseja que esses componentes possam se registrar como interessados e sejam avisados automaticamente quando a ordem mudar para `AUTORIZADA`, reduzindo acoplamento entre a ordem e cada reação concreta.

O padrão de projeto mais diretamente associado a essa solução é

- (A) Command, pois autorização é sempre representada como objeto executável em fila.
- (B) Builder, pois a ordem de pagamento deve ser construída passo a passo.

- (C) Proxy, pois toda notificação depende de controle de acesso remoto.
- (D) Flyweight, pois os observadores compartilham estado intrínseco para economizar memória.
- (E) Observer, pois interessados registrados são notificados quando ocorre mudança relevante de estado.

Questão 18

Uma classe abstrata ImportadorDocumento define o método importar(), que executa sempre a mesma sequência: validarArquivo(), extrairMetadados(), persistirDocumento() e registrarAuditoria(). As subclasses ImportadorPDF e ImportadorXML especializam apenas algumas etapas, mantendo inalterada a sequência geral do processo.

O padrão de projeto descrito é

- (A) Template Method, pois a superclasse define o esqueleto do algoritmo e permite especialização de etapas.
- (B) Iterator, pois a classe abstrata percorre uma coleção sem expor sua estrutura interna.
- (C) Mediator, pois centraliza a comunicação entre objetos independentes.
- (D) Prototype, pois os importadores são clonados a partir de um objeto base.
- (E) Chain of Responsibility, pois a requisição passa por tratadores até algum deles processá-la.

Questão 19

Durante a modelagem de um sistema de processos, uma equipe discutiu quem deveria calcular o prazo final de manifestação. A classe Processo possui as datas de distribuição, suspensão, reativação e intimação; a classe TelaProcesso apenas exibe informações; a classe Logger apenas registra eventos técnicos. Seguindo GRASP - General Responsibility Assignment Software Principles, um analista defendeu que a responsabilidade deveria ficar na classe que possui as informações necessárias para realizá-la.

O princípio GRASP invocado é

- (A) Controller, pois toda regra deve ficar no objeto que recebe eventos da interface.
- (B) Creator, pois a classe que cria objetos sempre calcula seus prazos.
- (C) Low Coupling, pois nenhuma classe pode depender de outra.
- (D) Information Expert, pois a responsabilidade é atribuída a quem detém os dados necessários.
- (E) Pure Fabrication, pois a responsabilidade deve ser retirada do domínio e colocada em classe artificial.

Questão 20

Em uma discussão de desenho de software, foram feitas as seguintes afirmações:

- I. Princípios SOLID orientam decisões de design de classes e módulos; padrões de projeto oferecem soluções recorrentes para problemas recorrentes de design.
- II. Padrões de projeto substituem princípios SOLID, pois ambos são mutuamente exclusivos e não devem ser usados no mesmo sistema.
- III. Um mesmo padrão de projeto pode apoiar a aplicação de um princípio; por exemplo, Strategy pode ajudar a reduzir condicionais e favorecer extensão de comportamento.

Está correto o que se afirma em

- (A) I, apenas.
- (B) I e III, apenas.
- (C) II e III, apenas.
- (D) I, II e III.
- (E) II, apenas.

12. Gabarito resumido

Gabarito

1-C; 2-A; 3-E; 4-B; 5-D; 6-A; 7-E; 8-C; 9-B; 10-D; 11-E; 12-A; 13-C; 14-B; 15-D; 16-C; 17-E; 18-A; 19-D; 20-B.

Questão	Gabarito	Tópico predominante
1	C	Abstração
2	A	Herança em Java
3	E	Encapsulamento
4	B	Polimorfismo
5	D	Sobrecarga x sobrescrita
6	A	SRP
7	E	OCP
8	C	LSP
9	B	ISP
10	D	DIP
11	E	Categorias GoF
12	A	Factory Method
13	C	Adapter
14	B	Facade
15	D	Strategy
16	C	State
17	E	Observer
18	A	Template Method
19	D	GRASP Information Expert
20	B	SOLID x padrões

13. Comentários completos

Questão 1 - Gabarito: C

Erro provável do candidato: confusão entre abstração e encapsulamento.

Alternativa A: Errada. Encapsulamento protege detalhes internos e controla acesso ao estado, mas o enunciado enfatiza seleção do que é essencial no modelo.

Alternativa B: Errada. Herança envolve especialização entre classes; o texto não afirma relação "é um" com DocumentoFisico.

Alternativa C: Correta. Abstração consiste em representar o essencial para determinado propósito e omitir detalhes sem relevância para o comportamento do sistema.

Alternativa D: Errada. Polimorfismo trata múltiplos comportamentos para uma mesma operação conforme o tipo concreto ou assinatura.

Alternativa E: Errada. Composição é relação todo-parte forte, não o ato de selecionar características essenciais.

Questão 2 - Gabarito: A

Erro provável do candidato: inversão da direção da herança.

Alternativa A: Correta. Se AnalistaSistemas extends ServidorPublico, AnalistaSistemas é subclasse/derivada/filha; ServidorPublico é superclasse/base/pai.

Alternativa B: Errada. Inverte a direção da herança.

Alternativa C: Errada. Java permite herança entre classes; apenas não permite herança múltipla de classes.

Alternativa D: Errada. Interface é implementada com implements; extends entre classes representa herança.

Alternativa E: Errada. Composição é relação todo-parte/tem-um, não generalização/especialização.

Questão 3 - Gabarito: E

Erro provável do candidato: conceito de encapsulamento.

Alternativa A: Errada. Java não permite herança múltipla de classes, e o problema não trata herança.

Alternativa B: Errada. Sobrecarga é uso de mesmo nome com assinaturas diferentes; não é requisito para encapsular.

Alternativa C: Errada. Abstração não elimina todo estado; ela seleciona aspectos essenciais.

Alternativa D: Errada. Polimorfismo paramétrico/generics não é o foco.

Alternativa E: Correta. O objetivo é ocultar o estado e permitir mudanças apenas por operações controladas, preservando regras do domínio.

Questão 4 - Gabarito: B

Erro provável do candidato: confusão entre polimorfismo, sobrecarga e composição.

Alternativa A: Errada. Agregação é relação todo-parte fraca; não é o fenômeno central descrito.

Alternativa B: Correta. O serviço usa uma abstração e o método calcular assume comportamento conforme a implementação concreta.

Alternativa C: Errada. Não há várias assinaturas de calcular; há implementações diferentes do mesmo contrato.

Alternativa D: Errada. Composição exigiria relação todo-parte forte com dependência de ciclo de vida.

Alternativa E: Errada. Serialização não tem relação com o comportamento descrito.

Questão 5 - Gabarito: D

Erro provável do candidato: confusão entre overloading e overriding.

Alternativa A: Errada. Mesmo nome não basta para sobrescrita; na primeira situação as assinaturas são diferentes na mesma classe.

Alternativa B: Errada. A segunda situação é redefinição de método herdado, típica sobrescrita.

Alternativa C: Errada. A primeira não é sobrescrita; e a segunda não é composição.

Alternativa D: Correta. Métodos de mesmo nome com assinaturas diferentes caracterizam sobrecarga; redefinir método herdado caracteriza sobrescrita.

Alternativa E: Errada. Não há generics nem relação todo-parte.

Questão 6 - Gabarito: A

Erro provável do candidato: identificação do princípio SOLID pelo cenário.

Alternativa A: Correta. A classe concentra várias responsabilidades e várias razões de mudança, violando SRP.

Alternativa B: Errada. OCP trata extensão sem modificação de módulos estáveis; não é o principal problema descrito.

Alternativa C: Errada. Não há cenário de subtipo quebrando contrato.

Alternativa D: Errada. Não se mencionou interface gorda obrigando clientes a depender de métodos inúteis.

Alternativa E: Errada. DIP recomenda depender de abstrações em pontos adequados, não proíbe toda classe concreta.

Questão 7 - Gabarito: E

Erro provável do candidato: confusão entre OCP e DIP.

Alternativa A: Errada. SRP não significa exatamente um método; a situação foca extensão sem mexer no código central.

Alternativa B: Errada. LSP não é o foco; não há violação de substitutibilidade descrita.

Alternativa C: Errada. ISP não exige sempre interfaces de um único método.

Alternativa D: Errada. DIP poderia estar relacionado à dependência de abstração, mas a ideia central do enunciado é aberto para extensão e fechado para modificação.

Alternativa E: Correta. OCP é exatamente a busca por ampliar comportamento com novas implementações, sem alterar continuamente o módulo estável.

Questão 8 - Gabarito: C

Erro provável do candidato: identificação de LSP.

Alternativa A: Errada. O número de métodos não define SRP.

Alternativa B: Errada. Criar extensão não é, por si só, violação de OCP.

Alternativa C: Correta. Se DocumentoTemporario não pode cumprir o contrato de Documento, ele não substitui corretamente o tipo base.

Alternativa D: Errada. ISP trata interfaces que forçam clientes a dependerem de métodos que não usam.

Alternativa E: Errada. DIP não tem relação com o fato de a exceção ser concreta.

Questão 9 - Gabarito: B

Erro provável do candidato: confusão entre ISP e SRP.

Alternativa A: Errada. OCP não proíbe novas classes.

Alternativa B: Correta. ISP recomenda interfaces específicas para que clientes não dependam de métodos irrelevantes.

Alternativa C: Errada. LSP trata substituição de subtipos mantendo contrato.

Alternativa D: Errada. DIP trata dependência de abstrações; e interfaces podem ter mais de um método quando coesas.

Alternativa E: Errada. SRP pode inspirar a decomposição, mas o problema explícito é interface gorda.

Questão 10 - Gabarito: D

Erro provável do candidato: confusão entre DIP e injeção de dependência.

Alternativa A: Errada. O número de construtores não define SRP.

Alternativa B: Errada. A mudança amplia substituição de infraestrutura, não restringe ao Oracle.

Alternativa C: Errada. LSP não é o foco.

Alternativa D: Correta. A regra de negócio passa a depender de uma abstração, com detalhes concretos dependendo do contrato.

Alternativa E: Errada. ISP não exige interfaces vazias.

Questão 11 - Gabarito: E

Erro provável do candidato: memorização das categorias GoF.

Alternativa A: Errada. Adapter e Facade são estruturais; Strategy e Observer são comportamentais.

Alternativa B: Errada. Singleton, Factory Method e Builder são criacionais.

Alternativa C: Errada. Nem todos são comportamentais.

Alternativa D: Errada. Observer é padrão GoF comportamental.

Alternativa E: Correta. A lista mistura criacionais (Singleton, Factory Method, Builder), estruturais (Adapter, Facade) e comportamentais (Strategy, Observer).

Questão 12 - Gabarito: A

Erro provável do candidato: identificação de padrão criacional.

Alternativa A: Correta. Factory Method encapsula a criação e reduz dependência do cliente em relação a classes concretas.

Alternativa B: Errada. Observer trata notificação de interessados após mudança de estado.

Alternativa C: Errada. Decorator adiciona responsabilidades a objetos sem alterar a classe original.

Alternativa D: Errada. Composite trata hierarquias todo-parte em árvore.

Alternativa E: Errada. Memento trata captura/restauração de estado.

Questão 13 - Gabarito: C

Erro provável do candidato: confusão entre Adapter e Facade.

Alternativa A: Errada. Facade simplifica subsistema, mas o ponto central aqui é compatibilizar interfaces incompatíveis.

Alternativa B: Errada. Proxy controla acesso a outro objeto; o foco não é controle, cache ou proteção.

Alternativa C: Correta. Adapter converte a interface de uma classe existente para a interface esperada pelo cliente.

Alternativa D: Errada. Builder trata construção progressiva de objeto complexo.

Alternativa E: Errada. Strategy trata família de algoritmos intercambiáveis.

Questão 14 - Gabarito: B

Erro provável do candidato: confusão entre Facade e Adapter.

Alternativa A: Errada. Não há conversão de interface incompatível; há simplificação de acesso.

Alternativa B: Correta. Facade oferece uma interface de alto nível para um subsistema complexo.

Alternativa C: Errada. Observer trata notificação de interessados; não é o foco.

Alternativa D: Errada. Prototype trata clonagem.

Alternativa E: Errada. Flyweight trata compartilhamento para economia de memória.

Questão 15 - Gabarito: D

Erro provável do candidato: identificação de Strategy.

Alternativa A: Errada. Singleton não resolve a escolha intercambiável de políticas de cálculo.

Alternativa B: Errada. Memento é para captura/restauração de estado.

Alternativa C: Errada. Composite é para estruturas árvore/todo-parte.

Alternativa D: Correta. Strategy encapsula algoritmos substituíveis e permite selecionar política em tempo de execução.

Alternativa E: Errada. Visitor adiciona operações sobre estrutura de objetos, não é o mais direto para políticas intercambiáveis simples.

Questão 16 - Gabarito: C

Erro provável do candidato: confusão entre Strategy e State.

Alternativa A: Errada. Strategy e State são parecidos, mas o enunciado enfatiza variação por estado interno e ciclo de estados.

Alternativa B: Errada. Template Method define esqueleto de algoritmo com passos especializados.

Alternativa C: Correta. State permite que um objeto altere seu comportamento quando seu estado interno muda.

Alternativa D: Errada. Abstract Factory cria famílias de objetos relacionados.

Alternativa E: Errada. Bridge separa abstração e implementação em hierarquias independentes.

Questão 17 - Gabarito: E

Erro provável do candidato: identificação de Observer.

Alternativa A: Errada. Command encapsula solicitação como objeto; o foco é notificação de interessados.

Alternativa B: Errada. Builder trata criação de objeto complexo.

Alternativa C: Errada. Proxy controla acesso.

Alternativa D: Errada. Flyweight economiza memória por compartilhamento.

Alternativa E: Correta. Observer define sujeito/observável e observadores notificados em mudanças de estado.

Questão 18 - Gabarito: A

Erro provável do candidato: identificação de Template Method.

Alternativa A: Correta. Template Method fixa a estrutura do algoritmo e delega etapas variáveis a subclasses.

Alternativa B: Errada. Iterator trata percorrer coleções.

Alternativa C: Errada. Mediator centraliza comunicação entre objetos.

Alternativa D: Errada. Prototype cria por clonagem.

Alternativa E: Errada. Chain of Responsibility envolve cadeia de tratadores, não esqueleto fixo de algoritmo.

Questão 19 - Gabarito: D

Erro provável do candidato: memorização de GRASP.

Alternativa A: Errada. Controller coordena eventos/casos de uso, mas não é a regra citada.

Alternativa B: Errada. Creator trata criação de objetos.

Alternativa C: Errada. Low Coupling busca reduzir acoplamento, não eliminar toda dependência.

Alternativa D: Correta. Information Expert atribui responsabilidade à classe que tem a informação necessária.

Alternativa E: Errada. Pure Fabrication cria classe artificial para melhorar design quando necessário, mas o enunciado aponta especialista da informação.

Questão 20 - Gabarito: B

Erro provável do candidato: pegadinha FGV sobre princípio versus padrão.

Alternativa A: Errada. A afirmativa III também está correta.

Alternativa B: Correta. SOLID e padrões não são excludentes; padrões podem ajudar a implementar princípios.

Alternativa C: Errada. A afirmativa II é falsa porque padrões e princípios podem coexistir.

Alternativa D: Errada. Inclui a afirmativa II, falsa.

Alternativa E: Errada. A afirmativa II é falsa.

14. Flashcards finais

Frente	Verso
O que é abstração?	Representar o essencial de uma entidade e omitir detalhes irrelevantes para o propósito do sistema.
O que é encapsulamento?	Controle do acesso ao estado interno e ocultação de detalhes de implementação.
O que é herança?	Relação de especialização em que uma subclasse herda características de uma superclasse.
O que é polimorfismo?	Capacidade de uma mesma operação ter comportamentos diferentes conforme o tipo/implementação.
Overloading x overriding?	Overloading: mesmo nome, assinaturas diferentes. Overriding: redefinição de método herdado.
Java permite herança múltipla de classes?	Não. Java permite uma superclasse direta, mas múltiplas interfaces.
SRP?	Uma classe/módulo deve ter uma responsabilidade coesa e uma razão principal para mudar.
OCP?	Aberto para extensão, fechado para modificação de código estável.
LSP?	Subtipos devem substituir tipos base sem quebrar o contrato esperado.
ISP?	Clientes não devem depender de métodos que não utilizam.
DIP?	Módulos de alto nível e baixo nível devem depender de abstrações, não de detalhes concretos.
Factory Method?	Encapsula/delega criação de objetos.
Abstract Factory?	Cria famílias de objetos relacionados sem especificar classes concretas.
Adapter?	Converte uma interface existente para outra esperada pelo cliente.
Facade?	Fornece interface simplificada para subsistema complexo.
Strategy?	Encapsula algoritmos intercambiáveis.
State?	Altera comportamento conforme estado interno.
Observer?	Notifica interessados registrados quando o sujeito muda de estado.
Template Method?	Define esqueleto do algoritmo e delega etapas a subclasses.
Information Expert?	Atribui responsabilidade à classe que possui as informações necessárias.

15. Checklist do que memorizar

- Diferença entre abstração e encapsulamento.
- Direção da herança: subclasse/derivada/filha versus superclasse/base/pai.
- Java: uma superclasse direta; múltiplas interfaces; Object como raiz implícita, exceto Object.
- Sobrecarga versus sobrescrita; ligação dinâmica em métodos de instância.
- Alta coesão e baixo acoplamento como objetivos de bom design.

- SOLID com nome expandido e palavra-chave de cada letra.
- LSP como substituição segura de subtipo por tipo base.
- DIP como dependência de abstrações; DI como técnica, não sinônimo.
- Categorias GoF: criacionais, estruturais e comportamentais.
- Factory Method x Abstract Factory; Adapter x Facade; Strategy x State; Decorator x Proxy.
- GRASP Information Expert como atribuição de responsabilidade a quem tem a informação.

16. Siglas, termos técnicos e mnemônicos

Sigla/termo	Expansão	Memorização
POO	Programação Orientada a Objetos	Paradigma de classes, objetos, estado e comportamento.
OOP	Object-Oriented Programming	Tradução técnica de POO.
SOLID	SRP, OCP, LSP, ISP, DIP	Cinco princípios de design orientado a objetos.
SRP	Single Responsibility Principle	Single = uma responsabilidade coesa.
OCP	Open/Closed Principle	Open para extensão; Closed para modificação.
LSP	Liskov Substitution Principle	Substitution = substituição segura.
ISP	Interface Segregation Principle	Segregation = separar interfaces.
DIP	Dependency Inversion Principle	Inverter dependência: concreto -> abstração.
DI	Dependency Injection	Técnica de fornecer dependências.
GoF	Gang of Four	Autores associados ao catálogo clássico de padrões.
GRASP	General Responsibility Assignment Software Principles	Princípios para atribuir responsabilidades.
API	Application Programming Interface	Contrato de comunicação entre sistemas/componentes.
CDI	Contexts and Dependency Injection	Mecanismo Java/Quarkus que viabiliza injeção de dependência.
DTO	Data Transfer Object	Objeto de transporte de dados.

17. Fontes técnicas e referências consultadas

As questões desta aula são autorais e não reproduzem questões reais. A estrutura dos enunciados foi inspirada no estilo de cadernos oficiais recentes da FGV para cargos de tecnologia, que apresentam cenários contextualizados, itens I/II/III, Verdadeiro/Falso e alternativas com conceitos próximos.

- Edital TJSC 2026 - Analista de Sistemas - FGV Conhecimento. Conteúdo programático de Engenharia de Software e Desenvolvimento.
- FGV Conhecimento - cadernos oficiais recentes de Analista de Sistemas e Desenvolvimento de Sistemas, usados apenas como referência de estilo de cobrança.
- Oracle Java Tutorials - Object-Oriented Programming Concepts; Inheritance.
- Microsoft Learn - .NET Architecture: aplicação de SOLID em camadas internas e desacoplamento.
- Gamma, Helm, Johnson e Vlissides - Design Patterns: Elements of Reusable Object-Oriented Software. Referência bibliográfica clássica para padrões GoF.
- Craig Larman - Applying UML and Patterns. Referência bibliográfica clássica para GRASP e atribuição de responsabilidades.